

Once you know the basics of JUnit, the next essential tool for robust unit testing in Java is **Mockito**, a popular **mocking framework**. It allows you to isolate the code you are testing (the System Under Test, or SUT) by providing fake, controllable dependencies for external objects (like databases, network services, or complex internal classes). Here is a step-by-step guide on Mockito, continuing from your basic JUnit tutorial.

Mockito Tutorial: Testing Dependencies

1. Why We Need Mockito

In real-world applications, one class (A) often uses another class (B) to do its job. In a unit test for **Class A**, you don't want to rely on the actual, live behavior of **Class B** because:

1. **Speed:** Class B might perform a slow database call or network operation.
2. **Isolation:** If Class B fails, the test for Class A fails, even if Class A's logic is perfect. This makes debugging difficult.
3. **Control:** You may need Class B to return a specific error or value that is hard to set up in a real environment.

Mockito solves this by letting you create a **mock** (a dummy object) for Class B, which you can program to return predictable results.

2. Project Setup: Mockito Dependencies

If you are using **JUnit 5**, you usually need the Mockito Core and the JUnit Jupiter integration.

If using Maven:

Add the following to your pom.xml (in addition to your JUnit dependencies):

```
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <version>5.5.0</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-junit-jupiter</artifactId>
  <version>5.5.0</version>
  <scope>test</scope>
</dependency>
```

If using Gradle:

Add the following to your build.gradle:

```
dependencies {
  // ... existing JUnit dependencies
  testImplementation 'org.mockito:mockito-core:5.5.0'
```

```
        testImplementation 'org.mockito:mockito-junit-jupiter:5.5.0'
    }
```

3. Example Scenario: Testing a Service Layer

We will test a UserService that relies on an external UserRepository to fetch data.

Step A: Create the Dependency Interface/Class

This is the class that performs external work (e.g., database access).

File: src/main/java/com/example/UserRepository.java

```
package com.example;

// Imagine this interface is implemented by a class that connects to a
// database
public interface UserRepository {
    String getUsernameById(int id);
}
```

Step B: Create the System Under Test (SUT)

This is the class we actually want to test. Its job is to process the data retrieved from the repository.

File: src/main/java/com/example/UserService.java

```
package com.example;

public class UserService {
    private final UserRepository userRepository;

    // Constructor Injection: The dependency is passed in
    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public String getFormattedUserName(int id) {
        // This is the line we want to control/fake
        String rawName = userRepository.getUsernameById(id);

        if (rawName == null) {
            return "User Not Found";
        }

        // This is the logic we want to test
        return "User: " + rawName.toUpperCase();
    }
}
```

```
}
```

4. Writing the Mockito Test

We will use Mockito to create a fake UserRepository and inject it into the UserService.

File: src/test/java/com/example/UserServiceTest.java

```
package com.example;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.when; // The critical Mockito method

// This annotation enables Mockito annotations in JUnit 5
@ExtendWith(MockitoExtension.class)
public class UserServiceTest {

    // 1. Define the MOCK object
    // @Mock creates a dummy version of the UserRepository.
    @Mock
    private UserRepository userRepositoryMock;

    // 2. Define the SUT (System Under Test)
    // @InjectMocks creates a UserService instance and automatically
    // injects the mock objects (userRepositoryMock) into its
    constructor.
    @InjectMocks
    private UserService userService;

    // --- TEST 1: Successful Path ---
    @Test
    void shouldReturnFormattedNameWhenUserExists() {
        // ARRANGE: Program the Mockito object!
        // When the mock's method is called with 100, return "Alice".
        when(userRepositoryMock.getUserNameById(100)).thenReturn("Alice");

        // ACT: Call the method on the SUT
        String result = userService.getFormattedUserName(100);

        // ASSERT: Verify the SUT's logic
        assertEquals("User: ALICE", result, "Name should be formatted");
    }
}
```

```

and capitalized.");
    }

    // --- TEST 2: Edge Case Path ---
    @Test
    void shouldReturnNotFoundWhenUserIsNull() {
        // ARRANGE: Program the Mockito object to return null

when(userRepositoryMock.getUserNameById(999)).thenReturn(null);

        // ACT
        String result = userService.getFormattedUserName(999);

        // ASSERT
        assertEquals("User Not Found", result, "Should handle null
return from repository.");
    }
}

```

5. Key Mockito Concepts

Concept	Annotation/Method	Purpose
Enabling Mockito	@ExtendWith(MockitoExtension.class)	Integrates Mockito with JUnit 5 tests.
Creating a Mock	@Mock	Creates a mock instance of a class or interface. It does not contain any real logic.
Injecting Mocks	@InjectMocks	Creates the actual class instance (SUT) and attempts to automatically inject the @Mock dependencies into it (via constructor, setter, or field).
Stubbing	when(...).thenReturn(...)	Programs the mock to return a specific value when a certain method is called with specific arguments. This isolates the test.
Verification	Mockito.verify(...)	(Advanced) Checks if a method on a mock was actually called during the test execution, ensuring the SUT interacted with its dependency as expected.